Reinforcement Learning Course: Week 1 Summary

Core Foundations: Tabular Methods and Mathematical Frameworks

1 Introduction

Week 1 covers the foundational mathematics and core concepts of Reinforcement Learning (RL), focusing on training agents to make decisions by interacting with their environment to maximize cumulative rewards. This document provides a comprehensive summary of foundational components, multi-armed bandits, Markov Decision Processes, value functions, and tabular algorithms.

2 Multi-Armed Bandits: The Foundations of Exploration

The Multi-Armed Bandit problem serves as a foundational example for understanding RL, representing a simplified decision-making scenario where an agent repeatedly chooses from K actions to maximize cumulative rewards over time.

- It requires the agent to balance exploration (gathering information about all actions) and exploitation (maximizing immediate rewards using the best-known action).
- Unlike supervised learning, RL involves evaluative feedback that only signals the quality of chosen actions rather than explicitly pointing to correct actions.
- Action values $q_*(a)$ are estimated through sampling averages or incremental update rules to refine estimates iteratively.

3 Markov Decision Processes (MDPs)

An MDP provides a framework for sequential decision-making in which actions affect immediate rewards as well as future outcomes. It is formally defined as a tuple:

$$M = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

- $\mathcal{S}$: State space, representing a specific condition or configuration of the environment at a given time.
- $\mathcal{A}$: Action space, representing the set of possible moves or decisions an agent can make.
- $\mathcal{P}(s'|s, a)$: Transition dynamics function, defining the probability of reaching a new state s' given state s and action a.
- $\mathcal{R}(s, a)$: Reward function, providing an objective at each time step and defining local and global goals.
- $\gamma \in [0, 1]$: Discount factor determining the current worth of future rewards.

3.1 Returns and Tasks

- For episodic tasks, the return G_t is the cumulative sum of rewards received from time step t onwards until termination.
- For continuing tasks without terminal states, the return G_t is defined as the discounted sum of future rewards: $G_t = \sum_{k=0}^{\infty} \gamma^k \mathcal{R}_{t+k+1}$.

4 Policies and Value Functions

Value functions estimate the expected return of an agent being in a certain state or performing an action under a policy π.

- **State-Value Function ($v_\pi(s)$):** The expected return starting from state s and following policy π thereafter.
- **Action-Value Function ($q_\pi(s, a)$):** The expected return starting from state s, taking action a, and following policy π thereafter.

The Bellman equation for v_π relates the value of a state to its potential successor states:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} \mathcal{P}(s', r|s, a) \left[r + \gamma v_\pi(s')\right]$$

5 Key Tabular Algorithms Implemented

5.1 1. Q-Learning (Off-Policy TD Control)

Q-Learning is a model-free, off-policy algorithm introduced by Watkins (1989) that learns the optimal action-value function independently of the agent's behavior policy. The update rule follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$$

5.2 2. SARSA (On-Policy TD Control)

SARSA is a model-free, on-policy algorithm that updates its action-value estimates based on the actions actually taken by the current policy. The update rule follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma Q(s', a') - Q(s, a)\right]$$

6 Exploration vs. Exploitation Strategies

To balance exploration and exploitation effectively, several strategies are employed:

- **Epsilon-Greedy (ϵ-greedy):** With probability ϵ, the agent chooses a random action for exploration; otherwise, it selects the greedy action for exploitation.
- **Upper Confidence Bound (UCB):** Balances exploration and exploitation by incorporating uncertainty into action selection.